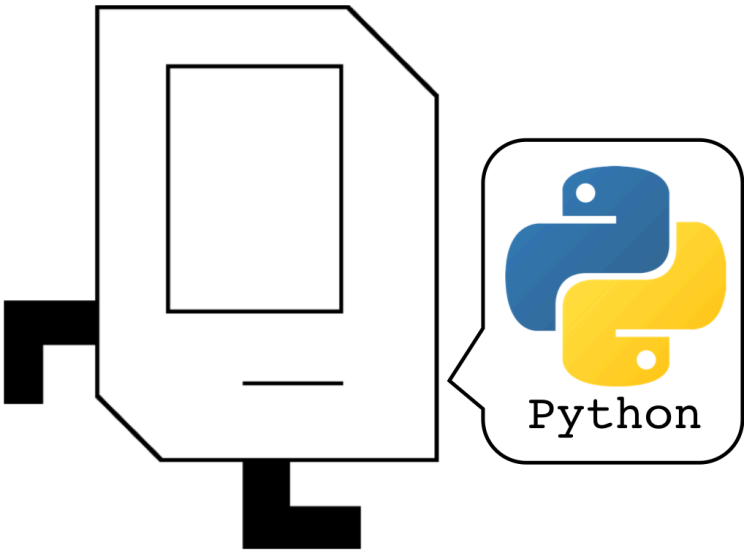
KAREL THE ROBOT LEARNS PYTHON



Chris Piech and Eric Roberts
Department of Computer Science
Stanford University
January 2019

Date of PDF Creation: April 2025

Chapter 1: Introducing Karel the Robot

In the 1970s, a Stanford graduate student named Rich Pattis decided that it would be easier to teach the fundamentals of programming if students could somehow learn the basic ideas in a simple environment free from the complexities that characterize most programming languages. Rich designed an introductory programming environment in which students teach a robot to solve simple problems. That robot was named Karel, after the Czech playwright Karel Čapek, whose 1923 play R.U.R. (Rossum's Universal Robots) gave the word robot to the English language.

Karel the Robot was quite a success. Karel has been used in introductory computer science courses all across the world and has been taught to millions of students. Many generations of Stanford students learned how programming works with Karel, and it is still the gentle introduction to coding used at Stanford.

What is Karel?

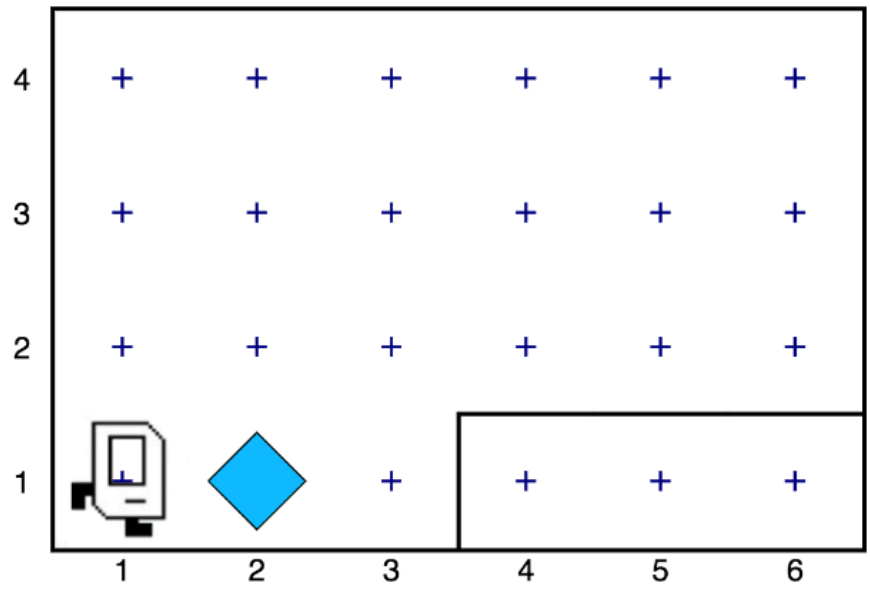
Karel is a very simple robot living in a very simple world. By giving Karel a set of commands, you can direct it to perform certain tasks within its world. The process of specifying those commands is called **programming**. Initially, Karel understands only a very small number of predefined commands, but an important part of the programming process is teaching Karel new commands that extend its capabilities.

Karel programs have much the same structure and involve the same fundamental elements as Python, a major programming language. The critical difference is that Karel's programming language is extremely small and as such the details are easy to master. Even so, you will discover that solving a problem can be challenging.

By starting with Karel, you can concentrate on solving problems from the very beginning. Problem solving is the essence of programming. And because Karel encourages imagination and creativity, you can have quite a lot of fun along the way.

Karel's World

Karel's world is defined by rows running horizontally (east-west) and columns running vertically (north-south). The intersection of a row and a column is called a corner. Karel can only be positioned on corners and must be facing one of the four standard compass directions (north, south, east, west). A sample Karel world is shown below. Here Karel is located at the corner of 1st row and 1st column, facing east.



Several other components of Karel's world can be seen in this example. The object in front of Karel is a beeper. As described in Rich Pattis's book, beepers are "plastic cones which emit a quiet beeping noise." Karel can only detect a beeper if it is on the same corner. The solid lines in the diagram are walls. Walls serve as barriers within Karel's world. Karel cannot walk through walls and must instead go around them. Karel's world is always bounded by walls along the edges, but the world may have different dimensions depending on the specific problem Karel needs to solve.

Karel's Commands

When Karel is shipped from the factory, it responds to a very small set of commands.

Here is what each command does:

Command	Description
`move()`	Asks Karel to move forward one block. Karel cannot respond to a `move()` command if there is a wall blocking its way.
`turn_left()`	Asks Karel to rotate 90 degrees to the left (counterclockwise).
`pick_beeper()`	Asks Karel to pick up one beeper from a corner and stores the beeper in its beeper bag, which can hold an infinite number of beepers. Karel cannot respond to a `pick_beeper()` command unless there is a beeper on the current corner.
`put_beeper()`	Asks Karel to take a beeper from its beeper bag and put it down on the current corner. Karel cannot respond to a `put_beeper()` command unless there are beepers in its beeper bag.

The empty pair of parentheses that appears in each of these commands is part of the common syntax shared by Karel and Python and is used to specify the invocation of the command. Eventually, the programs you write will include additional information in the space between the parentheses, but such information is not part of Karel's primitive world. These parentheses will therefore be empty in standard Karel programs, but you must remember to include them nonetheless.

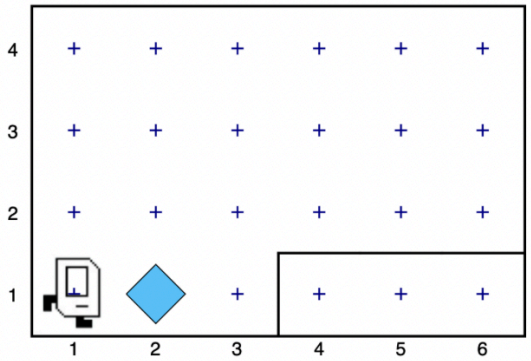
If Karel tries to do something illegal, such as moving through a wall or picking up a nonexistent beeper, an error condition occurs.

Karel’s commands are not executed on their own. Instead, you need to incorporate them into a Karel program. You will have a chance to see a few simple Karel programs in Chapter 2!

Chapter 2: Programming Karel

The simplest style of Karel program uses text to specify a sequence of built-in commands that should be executed when the program is **run**. Consider the simple Karel program below. The text on the left is the program. The state of Karel's world is shown on the right:

```
# File: FirstKarel.py
# ----------------------------
# The FirstKarel program defines a "main"
# function with three commands. These commands cause

# Karel to move forward one block, pick up a beeper
# and then move ahead to the next corner.
from karel.stanfordkarel import *

def main():
   move()
   pick_beeper()
   move()
```



Programs are typically written in a special application called an **Integrated Development Environment (IDE)** and most Karel programs are written in an IDE called PyCharm. Like an IDE, this reader has the ability to execute programs in order to help you *see* how things work as you learn.

The program is composed of several parts. The first part consists of the following lines:

```
# File: FirstKarel.py
# ----------------------------
# The FirstKarel program defines a "main"
# function with three commands. These commands cause
# Karel to move forward one block, pick up a beeper
# and then move ahead to the next corner.
```

These lines are an example of a **comment**, which is simply text designed to explain the operation of the program to human readers. Comments in both Karel and Python begin with the characters # and include the rest of the line. In a simple program, extensive comments may seem silly because the effect of the program is obvious, but they are extremely important as a means of documenting the design of larger, more complex programs.

The second part of the program is the line:

```
from karel.stanfordkarel import *
```

This line requests the inclusion of all definitions from the `karel.stanfordkarel` library. This library contains the basic definitions necessary for writing Karel programs, such as the definitions of the standard operations `move()` and `pick_beeper()`. Because you always need access to these operations, every Karel program you write will include this import command before you write the actual program.

The final part of the Karel program consists of the following function definition:

```
def main():
  move()
  pick_beeper()
  move()
```

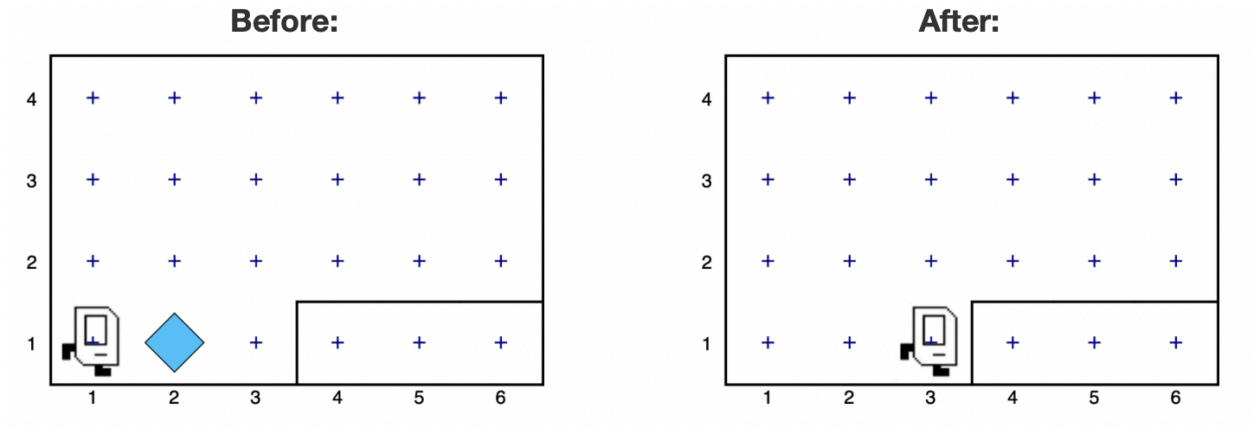
These lines represent the definition of a new **function**, which specifies the sequence of steps necessary to respond to a command. As in the case of the `FirstKarel` program itself, the function definition consists of two parts that can be considered separately: The first line constitutes the function header and the indented code following is the function body. If you ignore the body for now, the function definition looks like this:

```
def main():
  body of the function definition
```

The first word in the function header, `def`, is part of Python's syntactic structure. It says that you are creating a new function. The next word on the header line specifies the name of the new function, which in this case is **`main`**. Defining a function means that Karel can now respond to a new command with that name. The `main()` command plays a special role in a Karel program. When you start a Karel program it creates a new Karel instance, adds that Karel to a world that you specify, and then issues the `main()` command. The effect of running the program is defined by the body of the `main()` function, which is a sequence of commands that the robot will execute in order. For example, the body of the `main()` function for the `FirstKarel` program is:

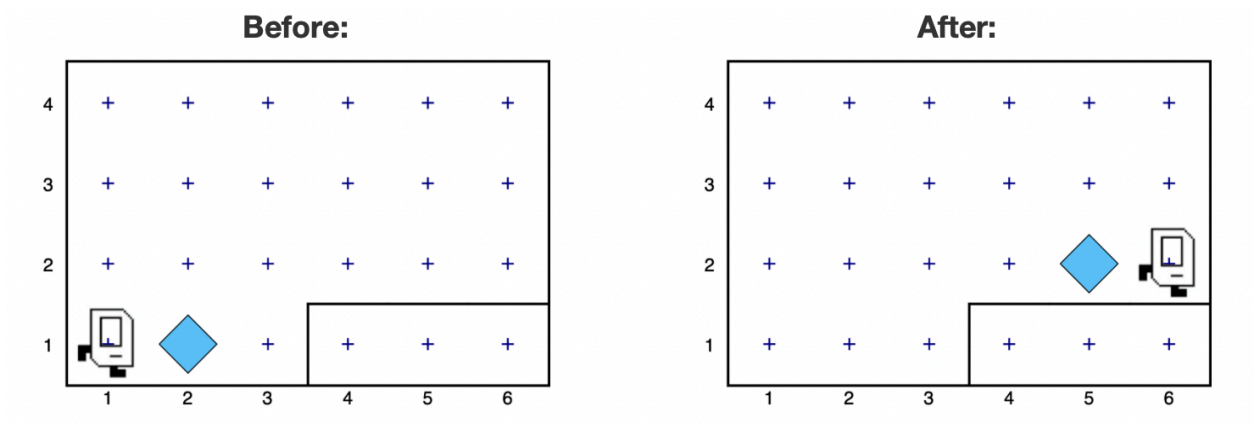
```
move()
pick_beeper()
move()
```

Thus, if the initial state of the world matches the example given in Chapter 1, Karel first moves forward into the corner containing the beeper, picks up that beeper, and finally moves forward to the corner just before the wall, as shown in the following before-and-after diagram:



Solving a more interesting problem

The `FirstKarel` program defined above doesn't do very much as yet. Let's try to make it a little more interesting. Suppose that the goal is not simply to get Karel to pick up the beeper but to move the beeper from its initial position on 2nd column and 1st row to the center of a ledge. Thus, your next assignment is to define a new Karel program that accomplishes the task illustrated in this diagram:

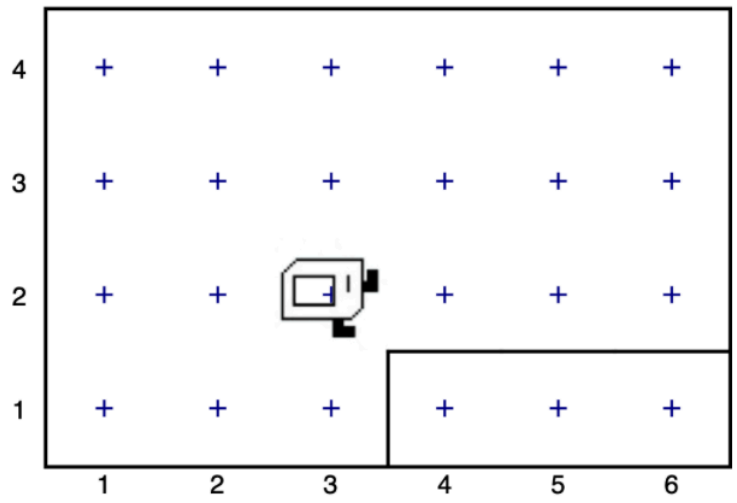


The first three commands in the new program—the ones that move forward, pick up the beeper, and then move up to the ledge—are the same as before:

```
move()
pick_beeper()
move()
```

From here, the next step is to turn left to begin climbing the ledge. That operation is easy, because Karel has a `turn_left()` command in its standard repertoire. Executing a `turn_left()` command at the end of the preceding sequence of commands leaves Karel facing north on the

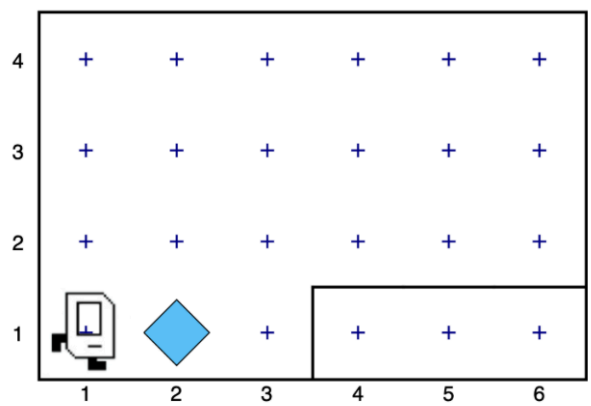
corner of 1st row and 3rd column. If Karel then executes a `move()` command, it will move north to reach the following position:

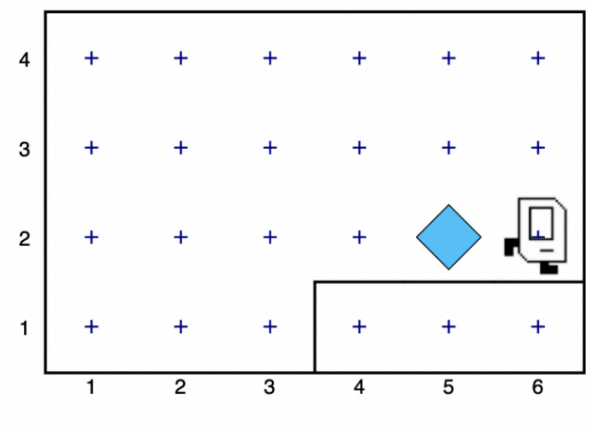


From here, the next thing you need to do is get Karel to turn right so that it is again facing east. While this operation is conceptually just as easy as getting Karel to turn left, there is a slight problem: Karel's language includes a `turn_left()` command, but no `turn_right()` command. It's as if you bought the economy model and have now discovered that it is missing some important features.

At this point, you have your first opportunity to begin thinking like a programmer. You have one set of commands, but not exactly the set you need. What can you do? Can you accomplish the effect of a `turn_right()` command using only the capabilities you have? The answer, of course, is yes. You can accomplish the effect of turning right by turning left three times. After three left turns, Karel will be facing in the desired direction. From here, all you need to do is program Karel to move over to the center of the ledge, drop the beeper and then move forward to the final position. Here is a complete implementation of a program that accomplishes the entire task:

```python
# File: FirstKarel.py
# ----------------------------
# The FirstKarel program defines a "main"
# function with commands that cause Karel to pick up
# a beeper and place it on a ledge.
from karel.stanfordkarel import *

def main():
   move()
   pick_beeper()
   move()
   turn_left()
   move()
   turn_left()
   turn_left()
   turn_left()
   move()
   move()
   put_beeper()
   move()
```





Chapter 3: Defining New Functions

Even though the `FirstKarel` program from the last chapter demonstrates that it is possible to perform the `turn_right()` operation using only Karel's built-in commands, the resulting program is not particularly clear conceptually. In your mental design of the program, Karel turns right when it reaches the top of the ledge. The fact that you have to use three `turn_left()` commands to do so is annoying. It would be much simpler if you could simply say `turn_right()` and have Karel understand this command. The resulting program would not only be shorter and easier to write, but also significantly easier to read.

Defining New Commands

Fortunately, the Karel programming language makes it possible to define new commands simply by including new function definitions. Whenever you have a sequence of Karel commands that performs some useful task--such as turning right--you can define a new function that executes that sequence of commands. The format for defining a new Karel function is much the same as the definition of `main()` in the preceding examples, which is a function definition in its own right. A typical function definition looks like this:

```
def name():
  commands that make up the body of the function
```

In this pattern, `name` represents the name you have chosen for the new function. To complete the definition, all you have to do is provide the sequence of commands in the lines after the colon, which are all indented by one tab. For example, you can define `turn_right()` as follows:

```
def turn_right():
  turn_left()
  turn_left()
  turn_left()
```

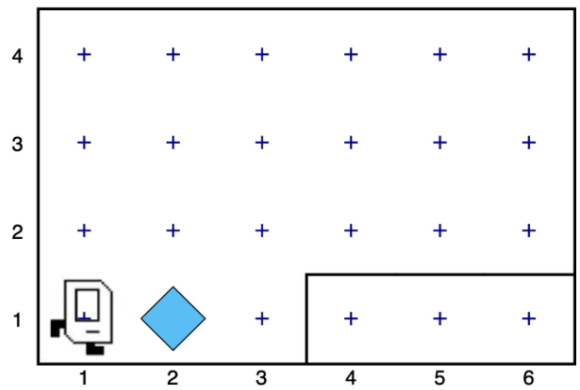
Similarly, you could define a new `turn_around()` function like this:

```
def turn_around():
  turn_left()
  turn_left()
```

You can use the name of a new function just like any of Karel's built-in commands. For example, once you have defined `turn_right()`, you could replace the three `turn_left()` commands in

the program with a single call to the `turn_right()` function. Here is a revised implementation of the program that uses `turn_right()`:

```
# File: BeeperPickingKarel.py
# ----------------------------
# The BeeperPickingKarel program defines a "main"
# function with three commands. These commands cause

# Karel to move forward one block, pick up a
# beeper and then move ahead to the next corner.
from karel.stanfordkarel import *
def main():
   move()
   pick_beeper()
   move()
   turn_left()
   move()
   turn_right()
   move()
   move()
   put_beeper()
   move()

def turn_right():
   turn_left()
   turn_left()
   turn_left()
```



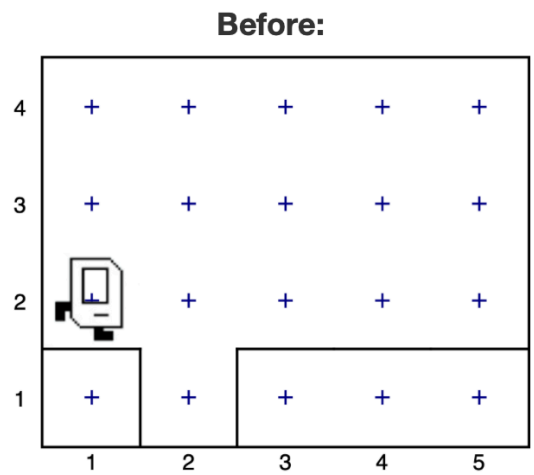
Function Code Blocks

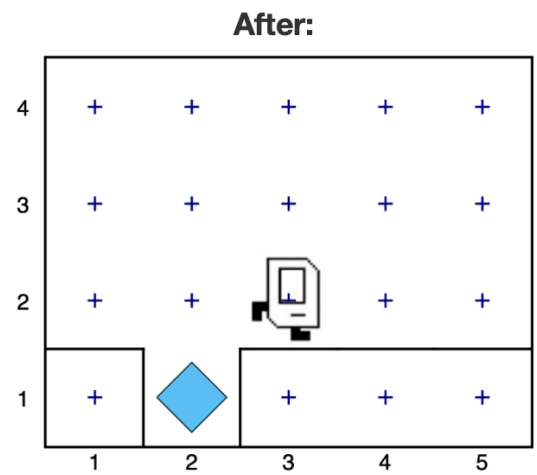
The group of commands that follow the colon '**:**', which are indented, are called a **code block**. The body of your function is a code block. Notice how the contents of a code block are all indented one tab in. This is important functionally since it allows Python to know what lines of code are in a given block.

You can define as many functions as you want. They should all be written one after another. You can't define a function inside another function.

Chapter 4: Decomposition

As a way of illustrating more of the power that comes with being able to define new functions, it's useful to have Karel do something a little more practical than move a beeper from one place to another. The roadways often seem to be in need of repair, and it might be fun to see if Karel can fill potholes in its abstract world. For example, imagine that Karel is standing on the "road" shown in the left-hand figure, one corner to the left of a pothole in the road. Karel's job is to fill the hole with a beeper and proceed to the next corner. The diagram on the right illustrates how the world should look after the program execution.





If you are limited to the four predefined commands, the `main()` function to solve this problem would look like this:

```
def main():
  move()
  turn_left()
  turn_left()
  turn_left()
  move()
  put_beeper()
  turn_left()
  turn_left()
  move()
  turn_left()
  turn_left()
  turn_left()
  move()
```

The initial motivation for defining the `turn_right()` function was that it was cumbersome to keep repeating three `turn_left()` commands to accomplish a right turn. Defining new functions has another important purpose beyond allowing you to avoid repeating the same command sequences every time you want to perform a particular task. The power to define functions unlocks the most important strategy in programming—the process of breaking a large problem down into smaller pieces that are easier to solve. The process of breaking a program down into smaller pieces is called **decomposition**, and the component parts of a large problem are called **subproblems**.

As an example, the problem of filling the hole in the roadway can be decomposed into the following subproblems:

1. Move up to the hole
2. Fill the hole by dropping a beeper into it
3. Move on to the next corner

If you think about the problem in this way, you can use function definitions to create a program that reflects your conception of the program structure. The main function would look like this:

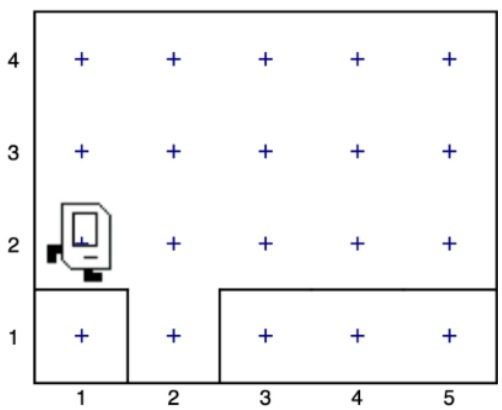
```
def main():
  move()
  fill_pothole()
  move()
```

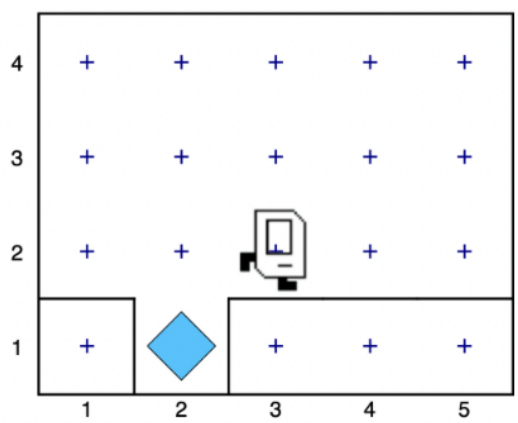
The correspondence with the outline is immediately clear, and everything would be great if only you could get Karel to understand what you mean by `fill_pothole()`. Given the power to define functions, implementing `fill_pothole()` is extremely simple. All you have to do is define a `fill_pothole()` function whose body consists of the commands you have already written to do the job, like this:

```
def fill_pothole():
  turn_right()
  move()
  put_beeper()
  turn_around()
  move()
  turn_right()
```

Here is the complete program. Notice how you can understand the programmer's intent simply from reading the `main()` function. When you run the program, the line highlighting shows how a computer will execute it, step by step. However, because the program is nicely broken down we can understand it on a human thought level:

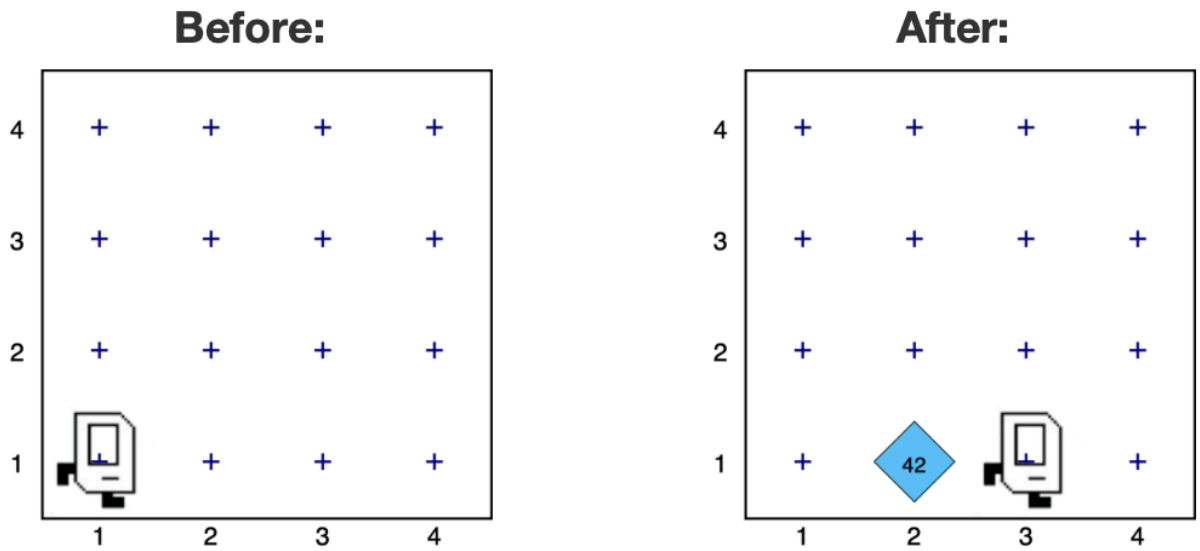
```python
from karel.stanfordkarel import *
def main():
    move()
    fill_pothole()
    move()

# Fills the pothole beneath Karel's current position by
# placing a beeper on that corner. For this function to work
# correctly, Karel must be facing east immediately above the
# pothole. When execution is complete, Karel will have
# returned to the same square and will again be facing east.
def fill_pothole():
    turn_right()
    move()
    put_beeper()
    turn_around()
    move()
    turn_right()

# Turns Karel 90 degrees to the right.
def turn_right():
    turn_left()
    turn_left()
    turn_left()

# Turns Karel around 180 degrees.
def turn_around():
    turn_left()
    turn_left()
```





Chapter 5: For Loops

One of the things that computers are especially good at is repetition. How can we convince Karel to execute a block of code multiple times? To see how repetition can be used, consider the task of placing 42 beepers:



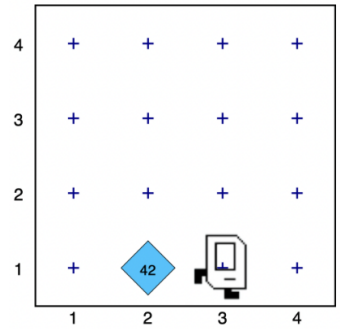
Basic For Loop

Since you know that there are exactly 42 beepers to place, the control statement that you need is a **`for` loop**, which specifies that you want to repeat some operation a fixed number of times. The structure of the `for` statement appears complicated primarily because it is actually much more powerful than anything Karel needs. The only version of the for syntax that Karel uses is:

```
for i in range(count):
  statements to be repeated
```

We will go over all the details of the `for` loop later in the class. For now you should read this line as a way to express, "repeat the statements in the function body *`count`* times." We can use this new **`for` loop** to place 42 beepers by replacing *`count`* with 42 and putting the command `put_beeper()` inside of the for loop code block. We call commands in the code block the **body**:

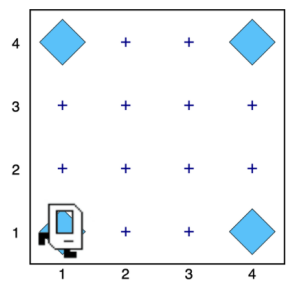
```python
# File: PlaceManyBeepers.py
# ----------------------------
# Places 42 beepers using a for loop
from karel.stanfordkarel import *

def main():
   move()
   # repeat put_beeper many times
   for i in range(42):
     put_beeper()
   move()
```



Matching Postconditions with Preconditions

The previous example gives the impression that a `for` loop repeats a single line of code. However the body of the `for` loop (the statements that get repeated) can be multiple lines. Here is an example of a program that puts a beeper in each corner of a world:

```python
# File: CornerBeepers.py
# ----------------------------
# Places one beeper in each corner
from karel.stanfordkarel import *
def main():
   # repeat the body 4 times
   for i in range(4):
      put_beeper()
      move()
      move()
      move()
      turn_left()
```



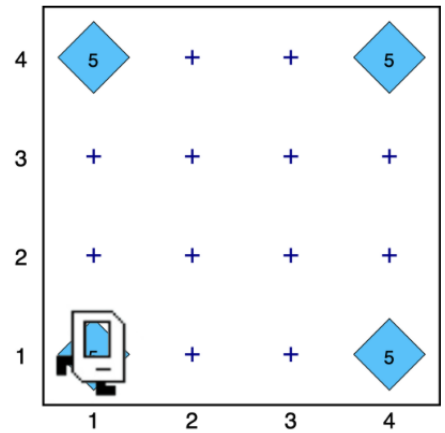
Pay very close attention to the way that the program flows through these control statements. The program runs through the set of commands in the `for` loop body one at a time. It repeats the body four times.

Perhaps the single most complicated part of writing a loop is that you need the state of the world at the end of the loop (the **postcondition**) to be a valid state of the world for the start of the loop (the **precondition**). In the above example the assumptions match. Good times. At the start of the loop, Karel is always on a square with no beepers facing the next empty corner. What if you deleted the `turn_left()` at the end of the loop? The postcondition at the end of the first iteration would no longer satisfy the assumptions made about Karel facing the next empty corner.

Nested Loops

Technically the body of a `for` loop can contain any control flow code, even other loops. Here is an example of a `for` loop that repeats a call to a function which also has a `for` loop. We call this a "nested" loop. Try to read through the program, and understand what it does, before running it:

```
# File: CornerFiveBeepers.py
# ----------------------------
# Places five beepers in each corner
from karel.stanfordkarel import *
def main():
   # Repeat once for each corner
   for i in range(4):
      put_five_beepers()
      move_to_next_corner()

# reposition karel to the next corner
def move_to_next_corner() :
   move()
   move()
   move()
   turn_left()

# places 5 beepers using a for loop
def put_five_beepers() :
   for i in range(5):
      put_beeper()
```

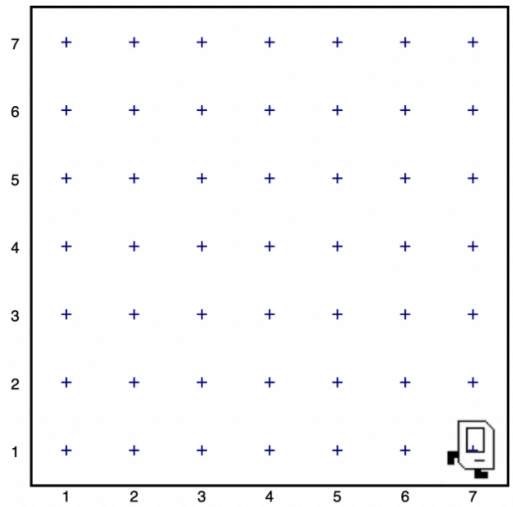


Chapter 6: While Loops

The technique of defining new functions, and defining for loops—as useful as they are—does not actually enable Karel to solve any new problems. Every time you run a program it always does exactly the same thing. Programs become a lot more useful when they can respond differently to different inputs.

As an example, let's say you wanted to write a program to have Karel move to a wall. But you don't simply want this program to work on one world with a fixed size. You would like to write a single program that could work on any world.

```
# File: MoveToWall.py
# -----------------------------
# Uses a "while" loop to move Karel until it hits
# a wall. Works on any sized world.
from karel.stanfordkarel import *

# the program starts with main
def main():
   # call the move to wall function
   move_to_wall()

# this is a very useful function
def move_to_wall():
   # repeat the body while the condition holds
   while front_is_clear():
      move()
```



For any sized world, Karel will move until it hits a wall. Notice that this feat can not be accomplished using a `for` loop. That would require us to know the size of the world at the time of programming.

Basic While Loop

In Karel, a `while` loop is used to repeat a body of code *as long as* a given condition holds. The `while` loop has the following general form:

```
while test:
   statements to be repeated
```

The control-flow of a `while` loop is as follows. When the program hits a `while` loop it starts repeating a process where it first *checks* if the test passes, and if so *runs* the code in the body.

When the program *checks* if the test passes, it decides if the `test` is true for the current state of the world. If so, the loop will run the code in the body. If the test fails, the loop is over and the program moves on.

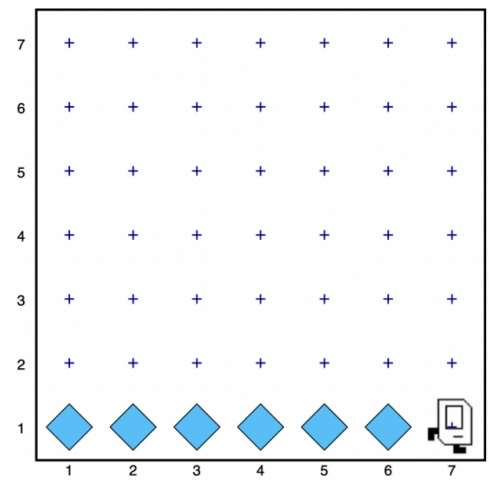
When the program *runs* the body of the loop, the program executes the lines in the body one at a time. When the program arrives at the end of the `while` loop, it jumps back to the top of the loop. It then rechecks the test, continuing to loop if it passes. The program does not exit the loop until it gets to a check, and the test fails.

Karel has many `test` statements, and we will go over all of them in the next chapter. For now we are going to use a single test statement: `front_is_clear()` which is true if there is no wall directly in front of Karel.

Fencepost Bug

Let's modify our program above to make it more interesting. Instead of just moving to a wall, have Karel place a line of beepers, one in each square. Again we want this program to work for a world of any size:

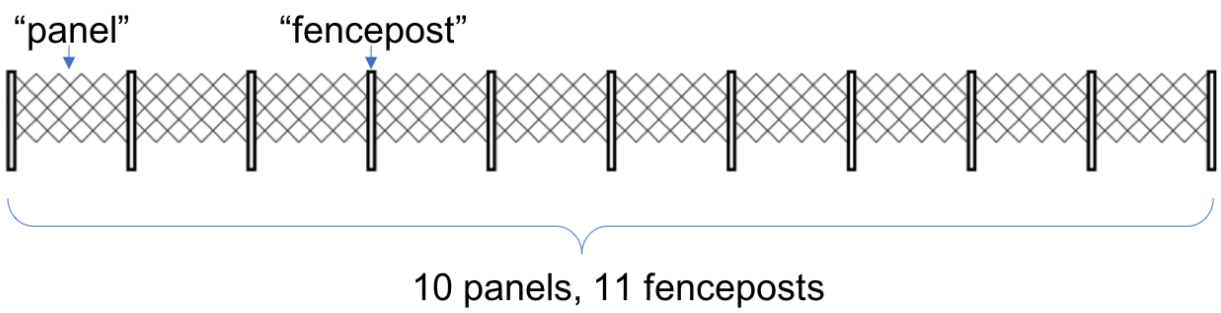
```
# File: BeeperLineBug.py
# ------------------------------
# Uses a while loop to place a line of beepers.
# This program works for a world of any size.
# However, because each world requires one fewer
# moves than put_beepers it always misses a beeper.
from karel.stanfordkarel import *

# program starts at main
def main():
    # repeats until karel faces a wall
    while front_is_clear():
        # place a beeper on current square
        put_beeper()
        # move to the next square
        move()
```



That looks great. Except for one problem. In every world, Karel doesn't place a beeper on the last square of the line (look closely). When Karel is on the last square, the program does not execute the body of the loop because the test no longer passes -- Karel is facing a wall. You might be tempted to try switching the order of the body so that Karel moves before placing a beeper.

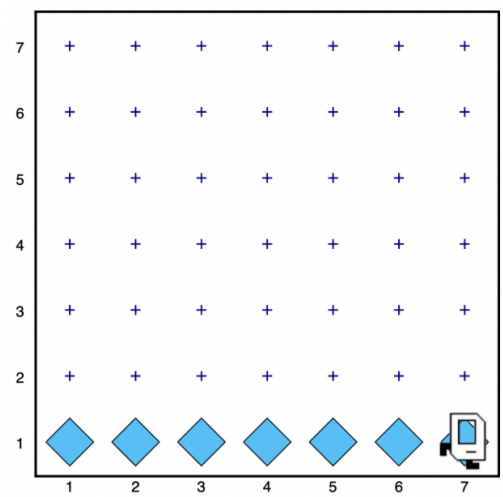
There is a deeper problem that no rearrangement of the body can solve. For the world with 7 columns, Karel needs to put 7 beepers, but should only `move()` 6 times. Since the `while` loop executes both lines when a test passes, how can you get the program to execute one command one more time than the other?

The bug in this program is an example of a programming problem called a **fencepost error**. The name comes from the fact that if you want to build a fence made of panels which have one fence post of any size, the number of fence posts is always one greater than the number of panels. How many fence posts, for example, do you need to build a fence with 10 panels? The answer is 11, as illustrated by the following diagram:



Once you discover it, fixing this bug is actually quite easy. Before Karel stops at the end of the world, all that the program has to do is place a final beeper:

```python
# File: BeeperLine.py
# ----------------------------
# Uses a while loop to place a line of beepers.
# This program works for a world of any size.
from karel.stanfordkarel import *

# program starts at main
def main():
    # repeats until karel faces a wall
    while front_is_clear():
        # place a beeper on current square
        put_beeper()
        # move to the next square
        move()
    # solves the fencepost bug
    put_beeper()
```



Chapter 7: Conditionals

The final core programming control-flow construct to learn are conditional statements (`if` and `if/else`).

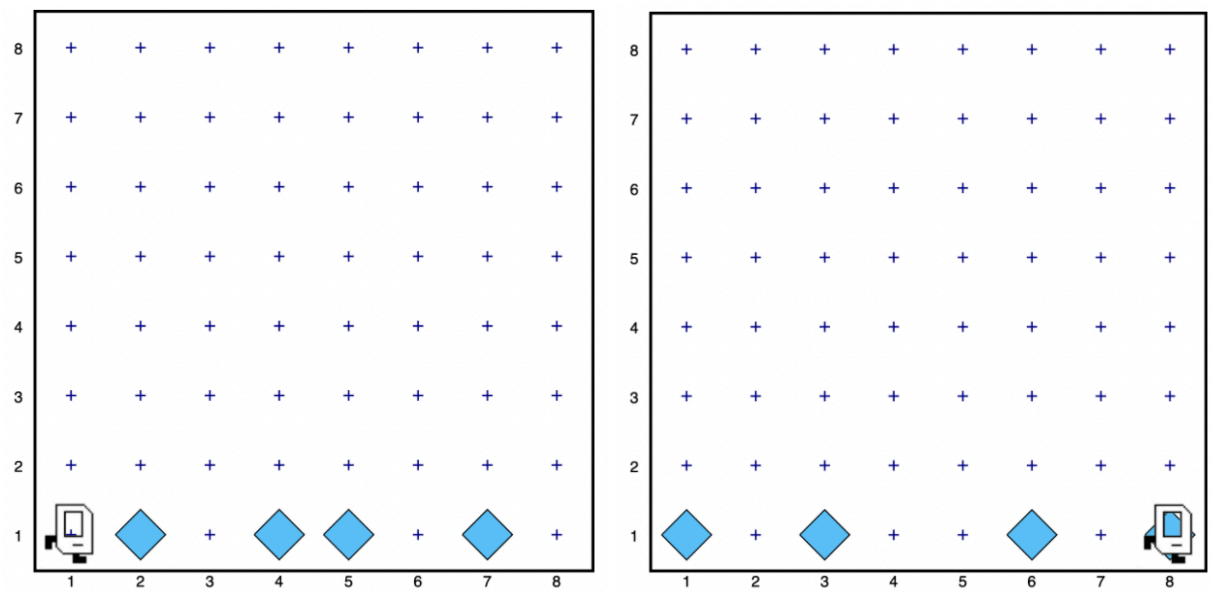
Basic Conditionals

An `if/else` statement executes an "`if`" code-block if and only if the provided *`test`* is true for the state of the world at the time the program reaches the statement. Otherwise the program executes the "`else`" code-block.

```
if test:
  if code-block
else:
  else code-block
```

To get a sense of where conditional statements might come in handy, let's write a program that has Karel invert a line of beepers. If a square previously had a beeper, Karel should pick it up. If a square has no beeper, Karel should put one down.

```python
from karel.stanfordkarel import *

# the start of the program
def main():
    while front_is_clear():
        invert_beeper()
        move()
    # to prevent a fencepost bug
    invert_beeper()

# picks up a beeper if one is present
# puts down a beeper otherwise
def invert_beeper():
    # an if/else statement
    if beepers_present():
        pick_beeper()
    else:
        put_beeper()
```



Note that an `if` statement does not need to have an `else` block -- in which case the statement operates like a `while` loop that only executes one time:

```python
if test:
    if code-block
```

Conditions

That last example used a new condition. Here is a list of all of the conditions that Karel knows of:

Test	Opposite	What it checks
`front_is_clear()`	`front_is_blocked()`	Is there a wall in front of Karel?
`beepers_present()`	`no_beepers_present()`	Are there beepers on this corner?
`left_is_clear()`	`left_is_blocked()`	Is there a wall to Karel's left?
`right_is_clear()`	`right_is_blocked()`	Is there a wall to Karel's right?
`beepers_in_bag()`	`no_beepers_in_bag()`	Does Karel have any beepers in its bag?
`facing_north()`	`not_facing_north()`	Is Karel facing north?
`facing_south()`	`not_facing_south()`	Is Karel facing south?
`facing_east()`	`not_facing_east()`	Is Karel facing east?
`facing_west()`	`not_facing_west()`	Is Karel facing west?

Putting it all together

Congrats! You now know all of the core programming control-flow blocks. While you learned them with Karel, methods, `while` loops, `for` loops, `if/else` statements work in the same way in almost all major languages, including Python.

Now that you have the building blocks you can put them together to build solutions to ever more complex problems. To a large extent, programming is the science of solving problems by computer. Because problems are often difficult, solutions—and the programs that implement

those solutions—can be difficult as well. In order to make it easier for you to develop those solutions, you need to adopt a methodology and discipline that reduces the level of that complexity to a manageable scale.

Chapter 8: Refinement

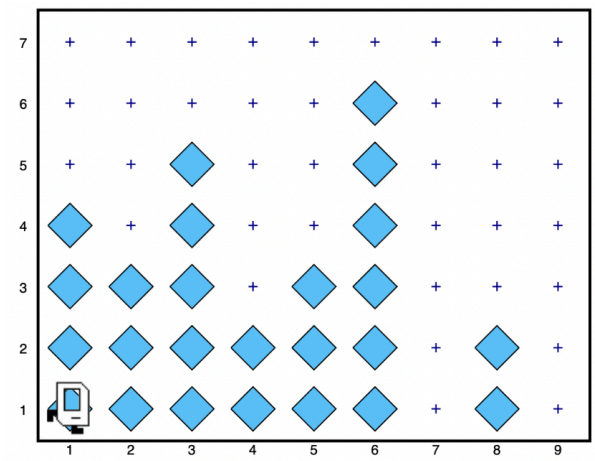
To a large extent, programming is the science of solving problems by computer. Because problems are often difficult, solutions—and the programs that implement those solutions—can be difficult as well. In order to make it easier for you to develop those solutions, you need to adopt a methodology and discipline that reduces the level of that complexity to a manageable scale.

In the early years of programming, the concept of computing as a science was more or less an experiment in wishful thinking. No one knew much about programming in those days, and few thought of it as an engineering discipline in the conventional sense. As programming matured, however, such a discipline began to emerge. The cornerstone of that discipline is the understanding that programming is done in a social environment in which programmers must work together. If you go into industry, you will almost certainly be one of many programmers working to develop a large program. That program, moreover, is almost certain to live on and require maintenance beyond its originally intended application. Someone will want the program to include some new feature or work in some different way. When that occurs, a new team of programmers must go in and make the necessary changes in the programs. If programs are written in an individual style with little or no commonality, getting everyone to work together productively is extremely difficult.

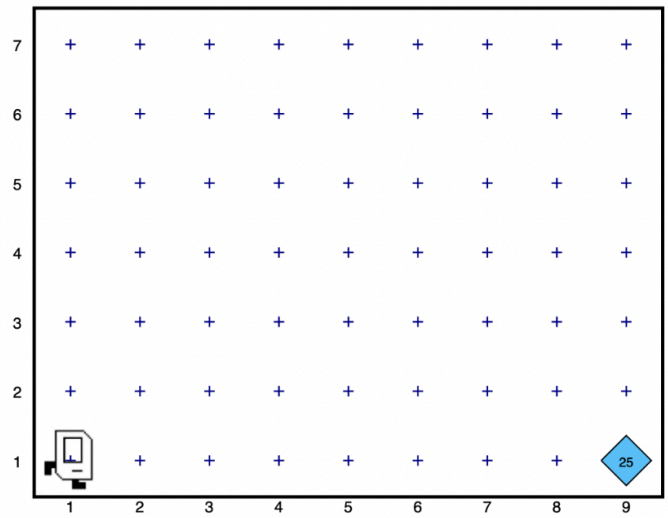
To combat this problem, programmers began to develop a set of programming methodologies that are collectively called **software engineering**. Using good software engineering skills not only makes it easier for other programmers to read and understand your programs, but also makes it easier for you to write those programs in the first place. One of the most important methodological advances to come out of software engineering is the strategy of **top-down design** or **stepwise refinement**, which consists of solving problems by starting with the problem as a whole. You break the whole problem down into pieces, and then solve each piece, breaking those down further if necessary. This top down strategy is complemented with **iterative testing** where you make sure that the smaller pieces of the solution are working before moving on.

An exercise in stepwise refinement

To illustrate the concept of stepwise refinement, let's teach Karel to solve a new problem. Imagine that Karel is now living in a world that looks something like this:



On each of the columns, there is a tower of beepers of an unknown height, although some columns (such as the 7th, and 9th in the sample world) may be empty. Karel's job is to collect all the beepers in each of these towers, put them back down on the easternmost corner of 1st row, and then return to its starting position. Thus, when Karel finishes its work in the example above, all 25 beepers currently in the towers should be stacked on the corner of 9th column and 1st row, as follows:



Importantly, you may assume that Karel initially *starts* with zero beepers in its bag. Each beeper picked up is added to its bag. When putting beepers in the corner, Karel can use the `beepers_in_bag()` test. We can also assume that the columns do not reach all the way up to the northmost wall.

The key to solving this problem is to decompose the program in the right way, while still being able to test as you go. This task is more complex than the others you have seen, which makes choosing appropriate subproblems more important to obtaining a successful solution.

The principle of top-down design

The key idea in stepwise refinement is that you should start the design of your program from the top, which refers to the level of the program that is conceptually highest and most abstract. At this level, the beeper tower problem is clearly divided into three independent phases. First, Karel has to collect all the beepers. Second, Karel has to deposit them at the last intersection. Third, Karel has to return to its home position. This conceptual decomposition of the problem suggests that the `main()` function for this program will have the following structure:

```
def main():
  collect_all_beepers()
  drop_all_beepers()
  return_home()
```

At this level, the problem is easy to understand. Of course, there are a few details left over in the form of functions that you have not yet written. Even so, it is important to look at each level of the decomposition and convince yourself that, as long as you believe that the functions you are about to write will solve the subproblems correctly, you will then have a solution to the problem as a whole.

Iterative testing as you go

Now that you have defined the structure for the program as a whole, it is time to move on to the first subproblem, which consists of collecting all the beepers. This task is itself more complicated than the simple problems from the preceding chapters. Collecting all the beepers means that you have to pick up the beepers in every tower until you get to the final corner. The fact that you need to repeat an operation for each tower suggests that you need a `while` loop here. The `while` loop will repeat the process of `collect_one_tower()`and then moving.

Caution: It is dangerous to try to write the entire program without **testing** it as you go. If you make a mistake it will be hard to find the mistake. We know that we are going to repeat the process of collecting one tower. Let us write and **test** collecting a single tower before we put the `collect_one_tower()` process in a `for` loop. Thus *temporarily* we can start with the following definition of `collect_all_beepers()`:

```
def collect_all_beepers() :
  # temporary implementation for testing purposes
  collect_one_tower()
  move()
```

As a guiding principle, if you have a complex loop, test the *body* of the loop before you write the entire loop.

Refining collect tower

When `collect_one_tower()` is called, Karel is either standing at the base of a tower of beepers or standing on an empty corner. In the former case, you need to collect the beepers in the tower. In the latter, you can simply move on. This situation sounds like an application for the `if` statement, in which you would write something like this:

```
if beepers_present():
  collect_actual_tower()
```

Before you add such a statement to the code, you should think about whether you need to make this test. Often, programs can be made much simpler by observing that cases that at first seem to be special can be treated in precisely the same way as the more general situation. In the current problem, what happens if you decide that there is a tower of beepers on every avenue but that some of those towers are zero beepers high? Making use of this insight simplifies the program because you no longer have to test whether there is a tower on a particular avenue.

The `collect_one_tower()` function is still complex enough that an additional level of decomposition is in order. To collect all the beeper s in a tower, Karel needs to undertake the following steps:

1. Turn left to face the beepers in the tower.
2. Collect all the beepers in the tower, stopping when no more beepers are found.
3. Turn around to face back toward the bottom of the world.
4. Return to the wall that represents the ground.
5. Turn left to be ready to move to the next corner.

Once again, this outline provides a model for the `collect_one_tower()` function, which looks like this:

```
def collect_one_tower():
  turn_left()
  collect_line_of_beepers()
  turn_around()
  move_to_wall()
  turn_left()
```

Function preconditions and postconditions

The `turn_left()` commands at the beginning and end of the `collect_one_tower()` function are both critical to the correctness of this program. When `collect_one_tower()` is called, Karel is always somewhere on the 1st row facing east. When it completes its operation, the program as a whole will work correctly only if Karel is again facing east at that same corner. Conditions that must be true before a function is called are referred to as **preconditions**; conditions that must apply after the function finishes are known as **postconditions**.

When you define a function, you will get into far less trouble if you write down exactly what the pre- and postconditions are. Once you have done so, you then need to make sure that the code you write always leaves the postconditions satisfied, assuming that the preconditions were satisfied to begin with. For example, think about what happens if you call `collect_one_tower()` when Karel is on the 1st row facing east. The first `turn_left()` function leaves Karel facing north, which means that Karel is properly aligned with the column of beepers representing the tower. The `collect_line_of_beepers()` function-which has yet to be written but nonetheless performs a task that you understand conceptually— simply moves without turning. Thus, at the end of the call to `collect_line_of_beepers()`, Karel will still be facing north. The `turn_around()` call therefore leaves Karel facing south. Like `collect_line_of_beepers()`, the `move_to_wall()` function does not involve any turns but instead simply moves until it hits the boundary wall. Because Karel is facing south, this boundary wall will be the one at the bottom of the screen, just below the 1st row. The final `turn_left()` command therefore leaves Karel on the 1st row facing east, which satisfies the postcondition.

Repeating the process

You run your program and it successfully clears one tower and leaves Karel in the promised postcondition. Wahoo! You have just hit a milestone in solving this hard task! We now have to repeat the process of clearing one tower using a `while` loop.

But what does this `while` loop look like? First of all, you should think about the conditional test. You want Karel to stop when it hits the wall at the end of the row. Thus, you want Karel to keep going as long as the space in front is clear. Thus, you know that the `collect_all_beepers()` function will include a `while` loop that uses the `front_is_clear()` test. At each position, you want Karel to collect all the beepers in the tower beginning on that corner. If you give that operation a name, which might be something like `collect_one_tower()`, you can go ahead and write a definition for the `collect_all_beepers()` function even though you haven't yet filled in the details.

You do, however, have to be careful. The code for `collect_all_beepers()` does not look like this:

```
def collect_all_beepers():
  # buggy loop!
  while front_is_clear():
    collect_one_tower()
    move()
```

This implementation is buggy for exactly the same reason that the first version of the general `Beeper Line` program from chapter 6 failed to do its job. There is a fencepost error in this version of the code, because Karel needs to test for the presence of a beeper tower on the last avenue. The correct implementation is:

```
def collect_all_beepers():
  while frontIsClear():
    collect_one_tower()
    move()
  collect_one_tower()
```

Note that this function has precisely the same structure as the main program from the `Place Beeper Line` program presented in chapter 6. The only difference is that this program calls `collect_one_tower()` where the other called `put_beeper()`. These two programs are each examples of a general strategy that looks like this:

```
def collect_all_beepers():
  while front_is_clear():
    perform some operation
    move()
  perform the same operation for the final corner
```

You can use this strategy whenever you need to perform an operation on every corner as you move along a path that ends at a wall. If you remember the general structure of this strategy, you can use it whenever you encounter a problem that requires such an operation. Reusable strategies of this sort come up frequently in programming and are referred to as **programming idioms** or **patterns**. The more patterns you know, the easier it will be for you to find one that fits a particular type of problem.

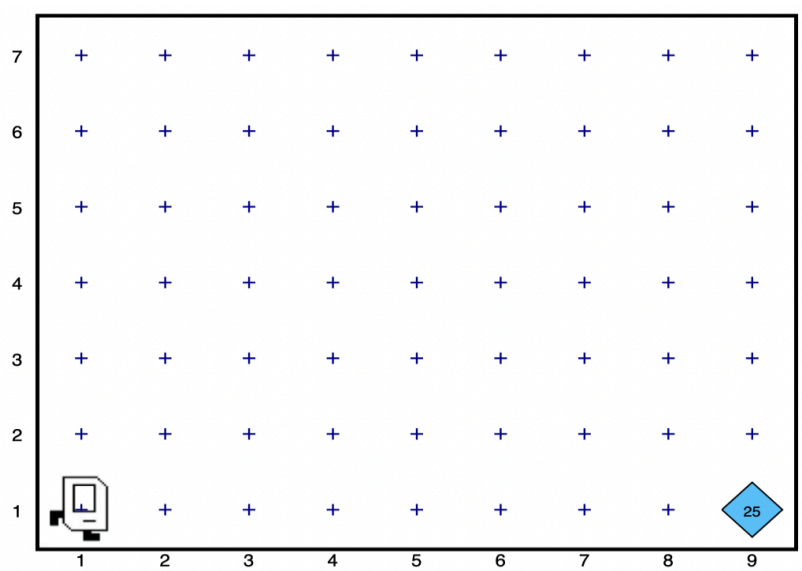
Finishing up

Although the hard work has been done, there are still several loose ends that need to be resolved. The main program calls two functions—`drop_all_beepers()` and `return_home()`— that are as yet unwritten. Similarly, `collect_one_tower()` calls `collect_line_of_beepers()` and `move_to_wall()`. Fortunately, all four of these functions are simple enough to code without any

further decomposition, particularly if you use `move_to_wall()` in the definition of `return_home()`. Here is the complete implementation:

```
# File: BeeperCollectingKarel.py
# -------------------------------
# The BeeperCollectingKarel class collects all the beepers
# in a series of vertical towers and deposits them at the
# eastmost corner on 1st row.
from karel.stanfordkarel import *

def main():
  collect_all_beepers()
  drop_all_beepers()
  return_home()

# Collects the beepers from every tower by moving along 1st
# row, calling collect_one_tower at every corner.  The
# postcondition for this function is that Karel is in the
# easternmost corner of 1st row facing east.
def collect_all_beepers():
  while front_is_clear():
    collect_one_tower()
    move()
  collect_one_tower()

# Collects the beepers in a single tower. When collect_one_tower
# is called, Karel must be on the 1st row facing east.  The
# postcondition for collect_one_tower is that Karel must again
# be facing east on that same corner.
def collect_one_tower():
  turn_left()
  collect_line_of_beepers()
  turn_around()
  move_to_wall()
  turn_left()

# Collects a consecutive line of beepers. The end of the beeper
# line is indicated by a corner that contains no beepers.
def collect_line_of_beepers():
  while beepers_present():
    pick_beeper()
    if front_is_clear():
      move()
```

```python
# Drops all the beepers on the current corner.
def drop_all_beepers() :
  while beepers_in_bag():
    put_beeper()

# Returns Karel to its initial position at the corner of 1st
# Avenue and 1st row, facing east.  The precondition for this
# function is that Karel must be facing east somewhere on 1st
# row, which is true at the conclusion of collect_all_beepers.
def return_home():
  turn_around()
  move_to_wall()
  turn_around()

# Moves Karel forward until it is blocked by a wall.
def move_to_wall():
  while front_is_clear():
    move()

# Turns Karel 180 degrees around
def turn_around():
  turn_left()
  turn_left()
```



Chapter 9: Extra Features

Painting corners

Karel also has the ability to paint corners in its world with the following colors:

- `"white"`, which removes any color on the square
- `"black"`
- `"blue"`
- `"red"`
- etc.

To use Karel's paint functionality, use the following commands:

Command	What it does
`paint_corner(COLOR_NAME)`	Karel colors the corner he's standing on `COLOR_NAME`. For example, to color the corner blue, you would write `paint_corner("blue")`.

Chapter 10: Reference

This appendix defines the structure of the Karel programming language on a single sheet.

Base Karel commands:

```
move()
turn_left()
put_beeper()
pick_beeper()
```

Karel program structures:

```
# Comments can be included in any part
# of a program. They start with a #
# and include the rest of the line.

def main() :
  code to execute

declarations of other functions
```

Names of the conditions:

`front_is_clear()`	`front_is_blocked()`
`beepers_present()`	`no_beepers_present()`
`beepers_in_bag()`	`no_beepers_in_bag()`
`left_is_clear()`	`left_is_blocked()`
`right_is_clear()`	`right_is_blocked()`
`facing_north()`	`not_facing_north()`
`facing_south()`	`not_facing_south()`
`facing_east()`	`not_facing_east()`
`facing_west()`	`not_facing_west()`

Conditions:

```
if condition:
  code run if condition passes
```

```
if condition:
  code block for "yes"
else:
  code block for "no"
```

Loops:

```
for i in range(count):
  code to repeat
```

```
while condition:
  code to repeat
```

Function Declaration:

```
def name():
  code in the body of the function.
```

Extra Karel Commands:

```
paint_corner(COLOR_NAME)
corner_color_is(COLOR_NAME)
```